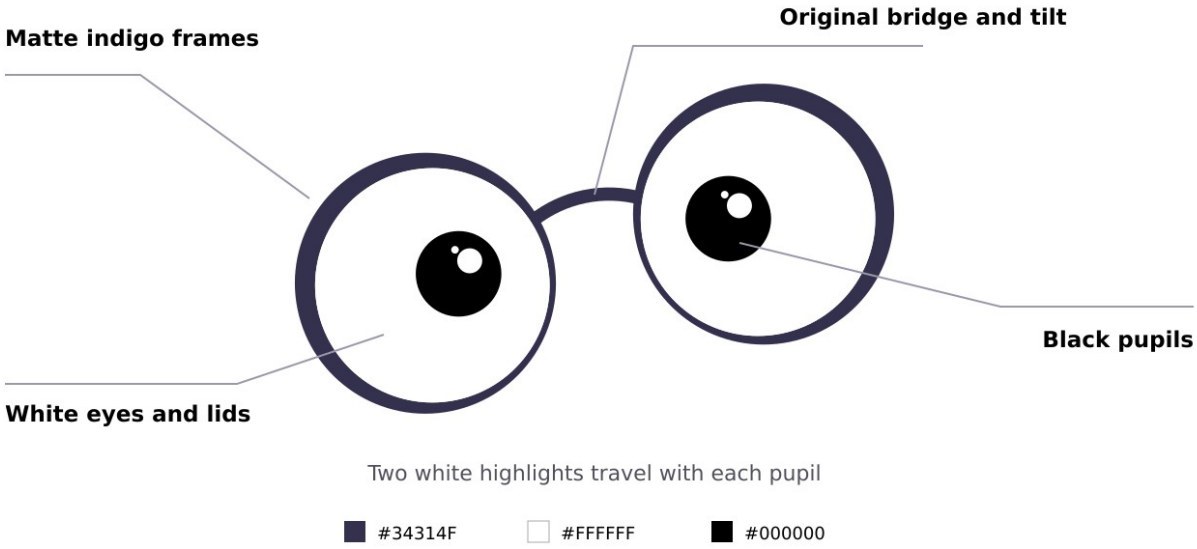
Geeky Animation Guide

Character design, editable motion and web and mobile integration

Use the editable vector rig as the master for new animations. Use the SVG and JavaScript player for interactive interfaces, and the supplied raster exports when a platform needs fixed frames. This guide keeps Geeky's original eyes and glasses recognizable while making the motion easier to extend.



Keep the character consistent

- Preserve the original contours, tilted silhouette and bridge. The matte motion edition uses dark indigo #34314F, white #FFFFFF and black #000000.
- Move each pupil together with its two highlights. Keep eyes white and pupils black; use white lids for blinks and expressions.
- For coil and orbit transformations, keep the complete original eyes visible. Let the frame shapes transform around them.
- Avoid metallic shading, bevels, reflections, extra facial features and bicycle or wheel transformations.

Start here

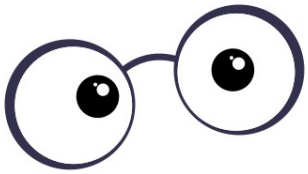
Preview the motion demo/index.html. Unzip the full kit first. Use the README launch instructions for the interactive demo.

Choose an integration README.md. Web and React examples use the player; mobile can use the self-contained WebView page or sprite sheets.

The DOCX and PDF contain stills. Open the SVG, GIF, HTML or MP4 files in a compatible viewer to see animation. Relative file links work from the unpacked kit and may be blocked by a document viewer's security settings.

The nine animation states

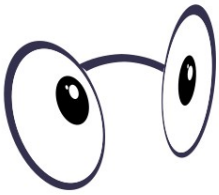
Each supplied state is a four-second cycle. The labels below are the visual meanings; the smaller text gives the exact state ID used in code and filenames. A paused thumbnail shows only one moment of the gesture.



Curious

idle

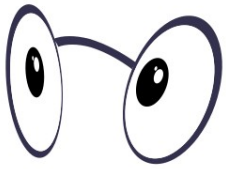
Glance and soft blink



Flutter right

running-right

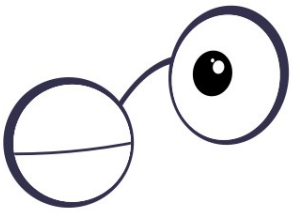
The frames lead a small glide



Flutter left

running-left

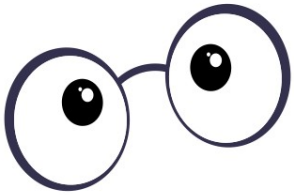
Bank gently back



Hello

waving

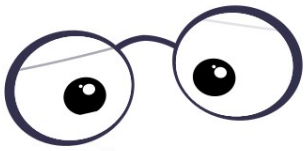
Lens wave with a wink



Delight

jumping

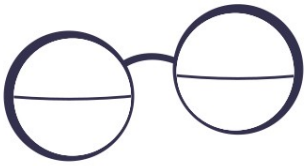
Crouch then lift and settle



Concern

failed

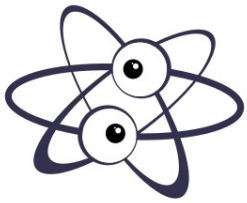
Downward gaze and worried tilt



Waiting

waiting

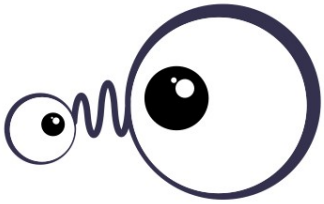
Patient sway becomes a doze



Orbit

running

Full eyes travel along the loops



Closer look

review

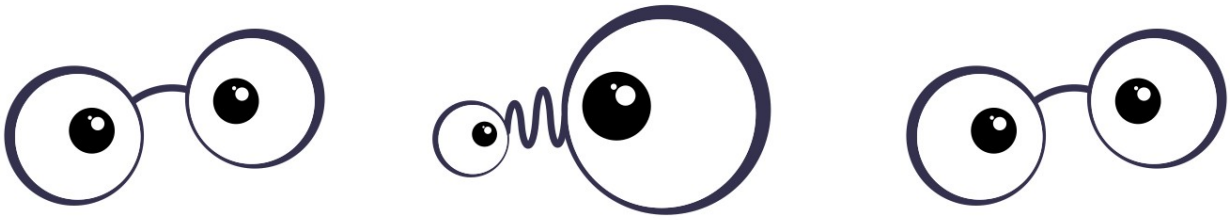
Coil plus one magnifying eye

Catalog stills are sampled from the editable matte rig. Ordinary blinks can briefly cover the pupils; the orbit and closer look transformations retain the original eye construction.

Use short event-driven gestures for greetings, completion and errors. Reserve repeating motion for a purposeful waiting state, with a visible pause or static alternative. The ID running selects the orbit transformation; running-right and running-left select the two flutters.

Build on the signature transformations

Closer look



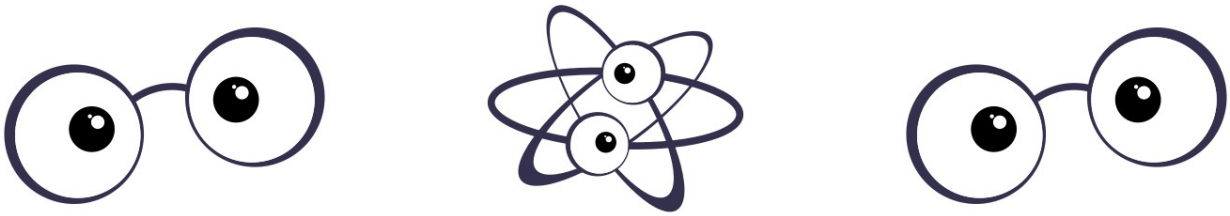
Start in the original pose

Transform with the eyes intact

Return to the original pose

State review turns the bridge into an elastic coil and enlarges one lens. The pupil, white eye and both highlights scale together. Use it for inspection, a preview or a moment of curiosity. Keep the coil flat and the eye readable at the largest scale.

Orbit



Start in the original pose

Transform with the eyes intact

Return to the original pose

State running stretches the lens frames into crossing loops while two complete eyes travel around the form. Keep the eyes white with black pupils and highlights; do not replace them with solid balls, wheels or generic dots. Leave room around the loops to avoid clipping.

Give a new gesture four clear beats

Beat	Design intent	Starting point
Anticipate	A glance, tilt or small compression hints at the action.	A brief, readable lead-in
Act	One dominant change carries the meaning.	Keep the eyes easy to track
Settle	A small overshoot or soft ease resolves the movement.	Reduce amplitude before stopping
Return	Rejoin the original silhouette without a jump.	Match the beginning pose

Treat the supplied timing as a reusable starting point. For a new gesture, change one main idea at a time and judge it at its actual UI size before adding secondary motion.

Choose the right file

Keep the vector rig and original SVG in version control. Exported SVG, GIF and sprite files are delivery formats; regenerating them is safer than editing every export independently.

Format	Use it for	Important limit
Source SVG and JavaScript	Changing shapes, timing, gaze and expression.	The rig owns animation; a vector editor does not automatically reconstruct that timing.
Animated SVG	Sharp, scalable looping motion with transparent surroundings.	Playback varies by viewer. Image embeds do not run SVG JavaScript.
Player and HTML	State changes, pause, speed, gaze and reduced motion.	Requires JavaScript and a browser or WebView.
GIF and animated WebP	Quick motion previews in compatible image viewers.	GIF has limited color and transparency. Neither exposes the player API.
Static SVG	A crisp fallback, icon or reduced-motion presentation.	One pose only.
PNG and WebP	Static UI assets and raster frame atlases.	Fixed resolution. Preserve alpha when exporting.
Sprite sheets and JSON	Native or game-style playback with predictable frame selection.	App code must handle frame timing, clipping and state changes.
MP4 showcase	Reviewing or presenting the complete motion sequence.	The supplied video has a background; it is not a transparent UI asset.
DOCX and PDF	Design decisions, annotated stills and integration reference.	No live animation in this guide.

Key files in the kit

Original artwork assets/original/Geeky-Original.svg

Editable rig src/geeky-motion.mjs

Player src/geeky-player.mjs

Asset and timing metadata manifest.json

Example exports assets/states/review/animation.svg

GIF preview assets/states/review/animation.gif

SVG embedded as an image has security restrictions, including disabled JavaScript. Use the player for interaction rather than relying on a scripted SVG in an img element.

MDN guidance on SVG image contexts • SVG animation elements

Edit and extend the motion

Work from one source of truth

1. Save a copy of the kit, keep the original SVG unchanged and commit the working version. Edit the rig rather than a GIF, MP4 or generated sprite sheet.
2. Choose the closest existing state. In statePose, the lens poses control position, rotation and scale; gaze offsets move pupils; lid values close the eyes; world moves the whole character.
3. Keep geometry and motion separate. Preserve the source path order and the eye clip paths. Moving or replacing a path without updating its group can detach highlights or make pupils escape the white eye.
4. Build the main action first, then anticipation and settling. Match the first and last poses of a loop. Inspect the halfway points as well as the extreme poses.
5. Add the new state ID to the state catalog and every state allowlist used by the player, demo and export scripts. Update the manifest and any reduced-motion fallback.
6. Regenerate the required formats using the kit README. Inspect alpha edges and the smallest display size, then test the actual application integration.

Understand the rig controls

Control	Changes	Protect
p	Two lens transforms: x, y, sx, sy and r.	The recognizable contours and bridge connections.
g	Per-eye pupil offsets.	Each pupil and its two highlights remain grouped.
lid	White eyelid closure for each eye.	Clipping within the eye and clear expression.
world	Translation, rotation and scale of the whole character.	Enough viewport space for motion extremes.
orbit and zoom	The signature loop and coil treatments.	The complete original eye construction.

Use gaze deliberately



Angles run clockwise from up: 0°, 90°, 180° and 270°. The kit includes 16 gaze directions. Call gaze(degrees) to override the pupils while keeping the current motion state; call gaze(null) to restore that state’s designed gaze.

A brief for the next animation

Specify the UI trigger, emotional meaning, one main movement, total duration, loop or one-shot behavior, return pose and reduced-motion alternative. Example: a single curious lean toward a newly opened detail, followed by a soft return to idle.

Integrate on the web

Use the JavaScript player when the UI needs to switch states or respond to events. Give its container a stable size and serve module examples over HTTP. The snippets assume an import beside the src folder; adjust the path in your app.

Vanilla JavaScript

Provide a sized element with id geeky and a button with id inspect. Clicking the button plays closer look once and returns to its final resting pose.

```
import { createGeeky } from "./src/geeky-player.mjs";

const geeky = createGeeky("#geeky", {
  state: "idle", reducedMotion: "system"
});
document.querySelector("#inspect").onclick = () =>
  geeky.playState("review", { loop: false });

// When removing this UI, call geeky.destroy().
```

React

Create the player after the element mounts and destroy it during cleanup. Keep the player in a ref; change states in a separate effect so prop changes do not create duplicate animation loops.

```
import { createElement, useEffect, useRef } from "react";
import { createGeeky } from "./src/geeky-player.mjs";

export function Geeky({ state = "idle" }) {
  const host = useRef(null), player = useRef(null);
  useEffect(() => {
    player.current = createGeeky(host.current,
      { reducedMotion: "system" });
    return () => player.current?.destroy();
  }, []);
  useEffect(() => {
    player.current?.playState(state);
  }, [state]);
  return createElement("div", { ref: host,
    style: { width: 128, height: 128 } });
}
```

Included React example examples/react/Geeky.js

React development Strict Mode can run setup and cleanup an extra time. Cleanup must stop the old player and remove its listeners. React effect lifecycle

Simple image embeds

For a looping SVG or GIF, use the supplied export as an image source and provide a static replacement when reduced motion is requested. An img does not expose the player API. Use empty alternative text for a purely decorative character; put meaningful product status in ordinary text nearby.

Full player contract docs/API.md

Use the kit in mobile apps

WebView is the closest match to the vector player

Bundle this self-contained page examples/mobile-webview/geeky-webview.html

Use the bundled page with iOS WKWebView, Android WebView or Flutter's webview_flutter. For React Native WebView, pass the bundled HTML through source.html; direct local-file loading has platform-specific limits. Enable JavaScript, wait for the ready event and send commands through GeekyBridge.receive. Restrict navigation to trusted content.

Examples of independent JavaScript calls from the native host after the page is ready

```
window.GeekyBridge.receive({
  type: "playState", state: "review", loop: false
});
window.GeekyBridge.receive({ type: "pause" });
window.GeekyBridge.receive({ type: "reducedMotion", value: true });
```

Pause when the app or screen goes into the background. Resume only when it is visible and motion is allowed. Send the app's reduced-motion setting through the bridge if the WebView does not inherit it reliably. Test transparency and scaling on real devices.

For Android local assets, follow WebViewAssetLoader guidance. Flutter's official webview_flutter package provides WebViewController and WebViewWidget; use the APIs for your installed version. These are integration starting points, not a claim of native-device validation.

Apple local file loading • React Native WebView guide

Native raster playback

Use the supplied PNG or WebP sprite atlas when a WebView is unsuitable. Crop the correct cell into a fixed-size viewport and advance from elapsed time rather than assuming every display frame arrives on schedule.

Property	Value
Cell	128 × 128 pixels with alpha
Each state	96 frames at 24 frames per second; four seconds
Atlas	8 columns × 12 rows; 1024 × 1536 pixels
Frame order	Row-major, left to right and then top to bottom
Loop index	floor(elapsedSeconds × 24) modulo 96
Crop origin	x = (index modulo 8) × 128; y = floor(index / 8) × 128

For a one-shot gesture, clamp the frame index to 95 instead of wrapping. Budget about 6 MiB for one decoded 1024 × 1536 RGBA atlas before renderer overhead, even if its compressed file is small. Load only the states the screen needs.

Native vector conversion requires extra work

A native SVG image library may render the static artwork without supporting SVG animation or the JavaScript rig. Lottie and Rive files are not supplied. Rebuilding the rig in either system requires mapping transforms, clipping, paths and timing, then comparing frames against the originals.

Make the motion comfortable and reliable

Accessibility and product behavior

- Honor reduced motion by default. Prefer a static meaningful pose for orbit, coil, flutter and jump rather than merely slowing large movement. Keep the product's status readable without animation.
- Offer pause or stop for persistent decorative loops. A four-second asset repeated indefinitely still continues beyond five seconds. WCAG 2.2.2 applies to qualifying moving content presented alongside other content.
- Use real buttons for interactive controls, with keyboard access and clear labels. Avoid announcing every frame to screen readers. Announce the underlying status change only when useful.
- Drive task completion from the application, not from an animation callback. Reduced motion freezes the player at a still pose, so its timeline completion callback does not run.
- Keep enough contrast around the dark frames on dark backgrounds. Test the smallest size and large zoom; the two highlights and eye outlines should remain legible.

MDN reduced-motion preference • W3C pause stop hide guidance

Before adding or shipping a state

- The initial and final poses join cleanly; the bridge, clips and highlights stay attached.
- Orbit and closer look retain the full original white eyes and black pupils.
- The whole movement fits the viewport on light, dark and transparent backgrounds.
- One-shot, loop, pause, resume, speed and state changes behave as intended.
- Reduced motion has a static fallback; a GIF or standalone SVG is swapped explicitly when needed.
- Hidden screens stop unnecessary work; player cleanup removes timers and listeners.
- Actual target browsers and devices have been checked, including multiple instances on one page.
- The manifest, examples, exports and documentation agree on state IDs, timing and dimensions.

Check the kit validation notes docs/TEST-REPORT.md. Distinguish automated export checks from browser rendering and real-device tests.

The app pet and the editable kit

The installed Geeky pet and this integration kit are separate deliverables. The pet uses the app's sprite runtime and state contract. This kit supplies editable vectors, a browser player and general-purpose exports; changing the kit does not automatically replace the installed pet.

Reference and provenance

The supplied Geeky SVG is the geometry source. The observed Instagram motion reference informed the idea of a recognizable silhouette transforming and returning, with anticipation and settling. Its crossing orbit loops were a useful direction; the bicycle treatment was excluded. The kit does not embed the reference video or its artwork.

The supplied YouTube reference did not play during review, so no motion-specific claims are based on it. Platform guidance was checked in October 2026; confirm behavior in your release targets.